DEVELOPING CODE · DOMAIN 1 - DEVELOPING CODE FOR DATA PROCESSING

Structured Streaming or declarative pipelines?

Section 1. A selection module. Eight scenarios, and the rules that decide them.

SECTION 1 of 10 WEIGHT 22% TOPIC Choosing

admin@sthithapragna.com

The default has flipped. Pipelines are now the recommended starting point, and **Structured Streaming is what you reach for when a pipeline cannot do the job.**

admin@sthithapragna.com

What a pipeline stops you having to build

- **Orchestration and lineage** across datasets, worked out from your queries rather than declared.
- **Checkpoint and state management**, handled rather than configured per query.
- **Expectations** for data quality, and **AUTO CDC** for slowly changing dimensions.
- **Enhanced autoscaling**. Ordinary autoscaling has known scale-down limitations for streaming.

> ON THE EXAM
>
> That last point is more than convenience. The production guidance for hand-written streaming is to disable autoscaling entirely.

admin@sthithapragna.com

Control against convenience

STRUCTURED STREAMING

You own every decision

- Any trigger, including real-time mode
- Any output mode, including complete
- Arbitrary sinks and arbitrary state
- You also own checkpoints and restarts

DECLARATIVE PIPELINES

The engine owns the plumbing

- Dependencies and lineage inferred
- Expectations and AUTO CDC built in
- Enhanced autoscaling and retries
- You accept the engine's constraints

HOW IT IS TESTED

Neither column is the safe default answer. The exam decides these on a specific capability the question quietly requires.

admin@sthithapragna.com

What pipelines cannot do

CONSTRAINT	CONSEQUENCE
Real-time mode is unsupported	Sub-second latency requirements rule pipelines out.
pivot() is unsupported	It needs eager loading to compute the output schema.
Time travel on materialized views	Not supported. Streaming tables only.
Iceberg reads	Cannot be enabled on materialized views or streaming tables.
External access	Published datasets are reachable only by Databricks clients.
A dataset is defined once	Only one operation may target it, unless append flows are used.

ON THE EXAM

These are the sentences a question hides inside a scenario. Learn them as triggers, not as trivia.

admin@sthithapragna.com

Real-time mode is Structured Streaming only

- **Not supported on pipelines**, classic or serverless. Not supported on plain serverless either.
- Runs on **dedicated** compute, and on **standard** compute for Python only.
- **Update output mode only.** Append and complete are not supported.
- Kafka, Event Hubs and Kinesis as sources. **Delta is not supported** as a source or a sink.

> ON THE EXAM
>
> So real-time mode is not simply a faster pipeline. It is a different set of constraints, and it excludes the lakehouse's own table format as a source.

admin@sthithapragna.com

What it will not do

- No **stream-to-stream joins**, no session windows, no deduplication within a watermark.
- Legacy arbitrary state operators are unsupported. **transformWithState works, with differences.**
- **Exactly-once sinks are not supported.** The sink mode must be set to at-least-once.
- Plain deduplication works, but its state is **unbounded**, so it grows without a watermark to evict it.

> ON THE EXAM
>
> A sub-second requirement combined with a stream-stream join is a contradiction. Notice when a question is asking for both.

admin@sthithapragna.com

Sinks brought arbitrary writes into pipelines

- Pipelines now have **sinks**, including a batch-level sink that runs your own Python for each micro-batch.
- That covers custom merge logic, writing to several destinations, and systems with no streaming write.
- **Python only.** SQL is not supported for sinks.
- **Only append flows can write to a sink**, and **expectations are not supported** for sinks.

> ON THE EXAM
>
> Older material says arbitrary sinks force you out of pipelines. That is no longer true, but the constraints matter.

admin@sthithapragna.com

Custom stateful logic in either place

- **transformWithState** is now recommended over the older map and flatMap state operators.
- It works in Structured Streaming **and inside a pipeline flow**, so custom state no longer forces a choice.
- Available from DBR 16.2. RocksDB became the default state store from DBR 17.3.
- Below that runtime, the RocksDB state store must be configured explicitly.

> ON THE EXAM
>
> Another gap that has closed. Needing custom state used to mean hand-written streaming; now it does not by itself.

admin@sthithapragna.com

Rules that settle most questions

IF THE SCENARIO REQUIRES	CHOOSE
Sub-second latency, or real-time mode	Structured Streaming
Multi-stage dependencies and lineage	Pipelines
Data quality expectations	Pipelines
SCD Type 1 or Type 2 handling	Pipelines, via AUTO CDC
Time travel on a derived aggregate	Structured Streaming, or a streaming table
Consumption by a non-Databricks client	Structured Streaming, or a pipeline sink
A new pipeline with no special constraint	Pipelines. That is the stated default.

ON THE EXAM

Work from the requirement to the tool, never the reverse. The final row is the tie-breaker when nothing else decides it.

admin@sthithapragna.com

What actually differs day to day

- Pipelines autoscale properly for streaming. Hand-written streams are told **not to autoscale at all**.
- Pipeline failures surface in an event log with dataset-level detail, rather than in cluster logs.
- Hand-written streams need continuous job scheduling, jobs compute and restart policies configured by you.
- Both are billed for compute. The difference is engineering time and the cost of mistakes.

ON THE EXAM

When a question mentions operational burden, retries, or a team spending time on plumbing, it is pointing at pipelines.

admin@sthithapragna.com

What migration actually involves

- There is **no in-place conversion**. A hand-written stream becomes a pipeline by being rewritten.
- Checkpoints do not transfer. The pipeline manages its own, so the new streaming table starts fresh.
- That makes source retention the first question, exactly as it was for a full refresh.
- A standalone dataset also cannot be moved into a pipeline. It has to be recreated there.

> ON THE EXAM
>
> Migration questions are really retention questions. If the source cannot replay the history, the migration needs a plan for it.

admin@sthithapragna.com

What to carry into the exam

- **Pipelines are the stated default** for new streaming work. Use that as the tie-breaker.
- **Real-time mode does not run on pipelines.** Sub-second latency is the clearest discriminator.
- Arbitrary sinks and custom state **no longer force** hand-written streaming. Both gaps have closed.
- Expectations are unsupported for sinks. Time travel is unsupported on materialized views.
- Migration means rewriting, and starting from a fresh checkpoint.

> ON THE EXAM
>
> Find the one capability the scenario requires, then check which tool lacks it. That is faster than weighing general merits.

admin@sthithapragna.com

> A payments team must flag suspicious card transactions and publish alerts to a Kafka topic within 400 milliseconds end to end. Events arrive from Kafka.
>
> The platform team's standard is to build all new streaming work as declarative pipelines.

What should the payments team do?

A Build it as a pipeline in continuous mode, with a short trigger interval.

B Build it as Structured Streaming in real-time mode, on dedicated compute.

C Build it as a pipeline, using a foreach batch sink to write to Kafka.

D Build it as a pipeline on serverless, which supports real-time mode.

◆ PAUSE AND DECIDE

admin@sthithapragna.com

What should the payments team do?

A. Build it as a pipeline in continuous mode, with a short trigger interval.

Continuous mode still runs micro-batches. It cannot reach 400 milliseconds.

B. Build it as Structured Streaming in real-time mode, on dedicated compute.

Real-time mode reaches around 300 milliseconds and is not available on pipelines. Kafka to Kafka is supported.

C. Build it as a pipeline, using a foreach batch sink to write to Kafka.

Solves the sink and not the latency. A sink does not make a pipeline sub-second.

D. Build it as a pipeline on serverless, which supports real-time mode.

Factually wrong. Real-time mode is unsupported on pipelines and on serverless.

ON THE EXAM

A team standard is not a requirement. When a stated latency target is unreachable by the standard approach, the target wins.

admin@sthithapragna.com

> A team is building a new bronze to gold pipeline over supplier data. It has four stages, needs record-level quality rules with rejected rows quarantined, and must apply Type 2 history to a supplier dimension.
>
> Latency requirements are relaxed: hourly is acceptable.

Which approach fits best?

A. Structured Streaming with foreachBatch, implementing merge logic and validation in Python.

B. A declarative pipeline using expectations and AUTO CDC.

C. Structured Streaming for bronze, then a pipeline for silver and gold.

D. Four standalone streaming tables, refreshed hourly on a schedule.

◆ PAUSE AND DECIDE

admin@sthithapragna.com

Which approach fits best?

A Structured Streaming with foreachBatch, implementing merge logic and validation in Python.

Hand-builds expectations and SCD handling that already exist, and inherits at-least-once semantics.

B A declarative pipeline using expectations and AUTO CDC.

Multi-stage dependencies, quality rules and Type 2 history are three separate pipeline strengths, with no constraint against them.

C Structured Streaming for bronze, then a pipeline for silver and gold.

Splits one pipeline in two and loses end-to-end lineage for no stated reason.

D Four standalone streaming tables, refreshed hourly on a schedule.

Standalone datasets have no orchestration between them, no expectations and no AUTO CDC.

ON THE EXAM

Three requirements each point the same way, and nothing points the other. That is the shape of an easy pipelines answer.

admin@sthithapragna.com

> An existing pipeline computes a daily revenue aggregate as a materialized view. Finance now needs to reproduce the exact figures published on any past date, for audit.
>
> The team proposes adding time travel queries against the materialized view.

What is the problem, and what should they do?

A Nothing is wrong. Time travel works on any Delta table, including materialized views.

B Time travel is unsupported on materialized views. Write the aggregate to a streaming table instead.

C Time travel requires change data feed to be enabled first on the materialized view.

D Materialized views only retain the current version, so they must be fully refreshed daily.

◆ PAUSE AND DECIDE

What is the problem, and what should they do?

A. Nothing is wrong. Time travel works on any Delta table, including materialized views.

Stated limitation: time travel is supported on streaming tables, not materialized views.

B. Time travel is unsupported on materialized views. Write the aggregate to a streaming table instead.

The limitation is specific to materialized views, and streaming tables do support history.

C. Time travel requires change data feed to be enabled first on the materialized view.

Different feature. Change data feed does not enable time travel.

D. Materialized views only retain the current version, so they must be fully refreshed daily.

Daily full refresh would not create queryable history either.

ON THE EXAM

Both dataset types are Unity Catalog managed tables, which makes A feel right. The limitation is specific and stated.

admin@sthithapragna.com

A pipeline enriches events and must also push each micro-batch into a third-party operational system that has no streaming connector, using a vendor Python client.

A senior engineer says this rules out declarative pipelines entirely and the team must rewrite in Structured Streaming.

Is that correct?

A Yes. Pipelines can only write to their own streaming tables and materialized views.

B No. A foreach batch sink runs custom Python per micro-batch, though expectations do not apply to sinks.

C No. Pipelines can write anywhere, and expectations still apply to the sink output.

D Yes, unless the third-party system supports Delta Sharing.

◆ PAUSE AND DECIDE

Is that correct?

A Yes. Pipelines can only write to their own streaming tables and materialized views.

This was true once. Sinks changed it, and older material still says this.

B No. A foreach batch sink runs custom Python per micro-batch, though expectations do not apply to sinks.

Correct on both halves: the capability exists, and the quality mechanism does not extend to it.

C No. Pipelines can write anywhere, and expectations still apply to the sink output.

Right conclusion, wrong reason. Expectations are explicitly unsupported for sinks.

D Yes, unless the third-party system supports Delta Sharing.

Delta Sharing is for sharing tables, not for writing into an operational system.

ON THE EXAM

C is the dangerous one. It reaches the correct conclusion through a false claim, which is exactly how a distractor is built.

admin@sthithapragna.com

A team runs a hand-written stream on an autoscaling cluster. Costs are high because the cluster rarely scales back down after nightly peaks, and one engineer spends most of a day each week on failed restarts and checkpoint issues.

The workload is a straightforward three-stage transformation. Latency of a few minutes is acceptable.

What is the strongest recommendation?

A Keep Structured Streaming, disable autoscaling, and size the cluster for peak.

B Rewrite as a declarative pipeline, gaining enhanced autoscaling and managed checkpoints.

C Keep Structured Streaming and move to real-time mode, which manages state automatically.

D Keep Structured Streaming and switch the trigger to availableNow to reduce cost.

◆ PAUSE AND DECIDE

admin@sthithapragna.com

What is the strongest recommendation?

A Keep Structured Streaming, disable autoscaling, and size the cluster for peak.

Follows the guidance and makes the cost worse, since peak sizing runs continuously.

B Rewrite as a declarative pipeline, gaining enhanced autoscaling and managed checkpoints.

Addresses both complaints at once, and nothing in the workload needs hand-written streaming.

C Keep Structured Streaming and move to real-time mode, which manages state automatically.

Real-time mode is for latency, adds constraints, and does not manage checkpoints for you.

D Keep Structured Streaming and switch the trigger to availableNow to reduce cost.

Helps cost, does nothing about the operational burden, and changes the workload to periodic batch.

ON THE EXAM

Two complaints, cost and operational burden, and one option addresses both. A addresses one by worsening the other.

admin@sthithapragna.com

A pipeline builds a wide reporting table. Part of the logic requires pivoting a long key-value table into columns, where the set of keys is not known in advance.

The pipeline fails when this step is added.

What is the cause and the best remedy?

A The pivot function is unsupported in pipelines. Compute the pivot outside and read the result in.

B The pivot must be applied to a materialized view rather than a streaming table.

C Pivots require the target schema to be declared explicitly on the dataset.

D The pipeline needs a full refresh to pick up the new columns.

◆ PAUSE AND DECIDE

admin@sthithapragna.com

What is the cause and the best remedy?

A The pivot function is unsupported in pipelines. Compute the pivot outside and read the result in.

A stated limitation. Pivot needs eager loading to compute the output schema, which pipelines do not do.

B The pivot must be applied to a materialized view rather than a streaming table.

The limitation is at pipeline level, not per dataset type.

C Pivots require the target schema to be declared explicitly on the dataset.

Declaring a schema does not restore the eager loading that pivot needs.

D The pipeline needs a full refresh to pick up the new columns.

Full refresh does not enable an unsupported operation.

ON THE EXAM

The reason matters: pivot needs to read the data to know its own output columns, and a declarative engine plans before it reads.

admin@sthithapragna.com

> An analytics team outside the Databricks estate needs to query a curated table produced by a pipeline, using their own Spark cluster and open source tooling. They report they cannot read the streaming table directly.
>
> The pipeline owners want to keep the pipeline as it is.

What is the right approach?

A Grant the external team SELECT on the streaming table in Unity Catalog.

B Use a pipeline sink to write a copy to an external Delta table the other team can read.

C Enable Iceberg reads on the streaming table so open source clients can query it.

D Rewrite the whole pipeline as Structured Streaming so the output is an ordinary Delta table.

◆ PAUSE AND DECIDE

admin@sthithapragna.com

What is the right approach?

A Grant the external team SELECT on the streaming table in Unity Catalog.

Permissions are not the barrier. Pipeline-published datasets are reachable only by Databricks clients.

B Use a pipeline sink to write a copy to an external Delta table the other team can read.

The documented route for external access, and it leaves the pipeline otherwise intact.

C Enable Iceberg reads on the streaming table so open source clients can query it.

Iceberg reads cannot be enabled on streaming tables or materialized views.

D Rewrite the whole pipeline as Structured Streaming so the output is an ordinary Delta table.

Would work, and discards a working pipeline when a sink solves it.

ON THE EXAM

Two stated limitations in one question: external clients cannot read published datasets, and Iceberg reads cannot be enabled on them.

admin@sthithapragna.com

A team plans to migrate a hand-written stream to a declarative pipeline. The stream reads from a Kafka topic with seven days of retention and has been running for two years, maintaining a streaming table of 3 billion rows.

They expect to switch over with no gap in the data.

No constraint in the workload requires hand-written streaming.

What must they plan for?

A Nothing special. The pipeline can adopt the existing checkpoint and continue where the stream left off.

B The pipeline starts from a fresh checkpoint, so history beyond seven days must be backfilled separately.

C The migration is impossible; a stream with existing state cannot become a pipeline.

D They must run a full refresh on the existing streaming table before migrating.

◆ PAUSE AND DECIDE

What must they plan for?

A Nothing special. The pipeline can adopt the existing checkpoint and continue where the stream left off.

Checkpoints do not transfer. The pipeline manages its own.

B The pipeline starts from a fresh checkpoint, so history beyond seven days must be backfilled separately.

Kafka retains seven days against two years of history. The gap has to be filled from the existing table.

C The migration is impossible; a stream with existing state cannot become a pipeline.

It is a rewrite, not an impossibility.

D They must run a full refresh on the existing streaming table before migrating.

Would destroy the very history that needs preserving, since Kafka holds only seven days.

ON THE EXAM

The same retention arithmetic as module 5. Two years of history behind a seven-day source means the difference must come from somewhere else.

admin@sthithapragna.com